

write a piece of code that holds HTML elements in it, you can write it within the self-executing function. Given below is an example of this:

```
(function()
{
    //function code
})(arguments)
```



Note: All the examples in this article are according to the ECMAScript5+. Make sure your browsers implement the same.

Functions as 'first class citizens'

This is a very common term in JavaScript for functions. You will come across this term on the Web many times while searching for JavaScript. Functions are considered first class citizens because they possess the following properties:

1. Are assigned to a variable
2. Can be passed as an argument
3. Can be returned from another function

```
//Can be passed as an argument
function sum(a,b)
{
    return a+b;
}

sum(add(5,6));
```

```
//Can be returned from another function
function sum(a,b)
{
    return function(a,b)
    {
        return a+b;
    }
}
```

Scope

The scope of the variable is quite important in JavaScript and it is slightly different from the normal programming languages.

Local variable: If a variable is declared with the keyword 'var' inside the function, we call it a local variable. The scope of the variable is limited to that function only.

Global variable: If you declare any variable without the keyword 'var', it is considered as a global variable. That variable will be the part of your root object, i.e., a window object. Memory will not be released after its execution. So it is not advisable to use global variables for any reason.



Note: If you declare any variable with 'var' outside the function, it will also be considered as a global variable.

An example

```
//code
name="Open Source For You"
function info()
{
    var name="Electronics For You";
    console.log(name) //prints Electronics For You
}

console.log(name); //prints Open Source For You
```

Closures

You can also use a function within a function. That's right! This is possible in JavaScript. Any function (parent) can contain another child function.

```
function parent()
{
    function child()
    {
        // child will have access to all the parent variables
    }
}
```

The good part is that all the variables declared within your parent function can also be accessed by your child function.

Closure is a separate space in the memory that will be created whenever a child function resides inside the parent function. Closures will contain all the variables of the parent function, so whenever the child gets access to that variable, it can start its execution. In simple terms, it's like first the parent execution will take place and then the child execution. Memory of the parent function will not be destroyed after its execution. It will be still available for its child function.

An example of closure

```
function info()
{
    var magazineName="Open Source For You";
    var name=function()
    {
        console.log(name);
    }
    return name();
}

info() //will print Open Source For You
```